



Algorithmen und Datenstrukturen

Leon Johann Brettin

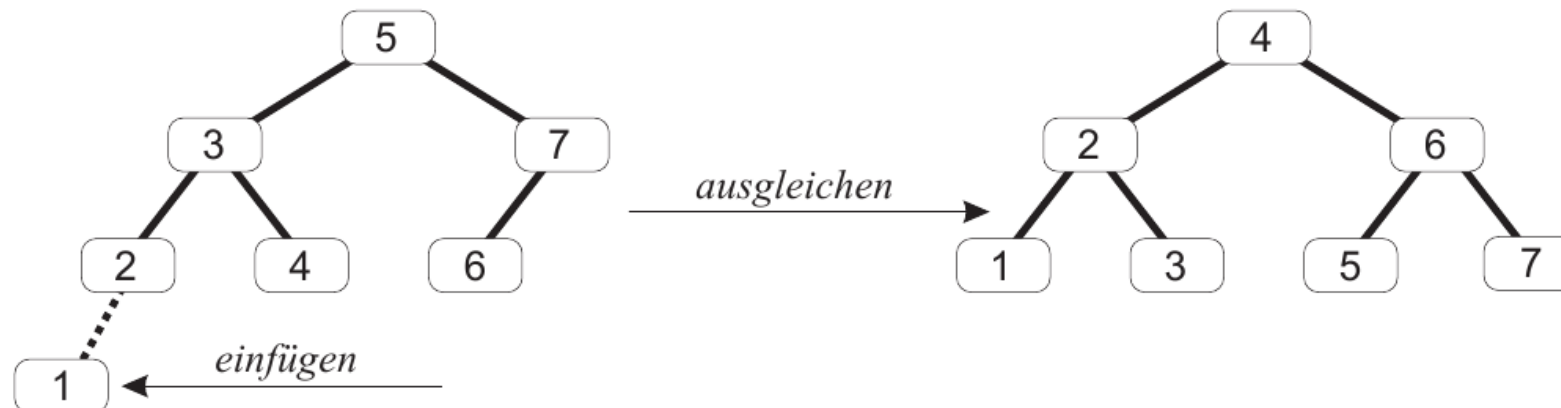


Algorithmen und Datenstrukturen

10. Bäume 2

Ausgeglichene Bäume

- Wie verhindert man, dass Suchbäume entarten?
- Jeweils ausgleichen ist zu aufwendig: Im Beispiel fügen wir den Knoten 1 ein und gleichen den Baum aus.
- Das Beispiel zeigt, dass beim Ausgleichen eventuell jeder Knoten bewegt werden müsste $\rightsquigarrow$ wieder $O(n)$!





Lösungsideen

- abgeschwächtes Kriterium für ausgeglichene Höhe
 - Beispiel: AVL Bäume
- ausgeglichene Höhe, aber unausgeglichener Verzweigungsgrad
 - direkte Realisierung als Mehrwegbäume, Beispiel: B-Bäume
- Kodierung als binäre Bäume
 - Beispiel: Rot-Schwarz-Bäume

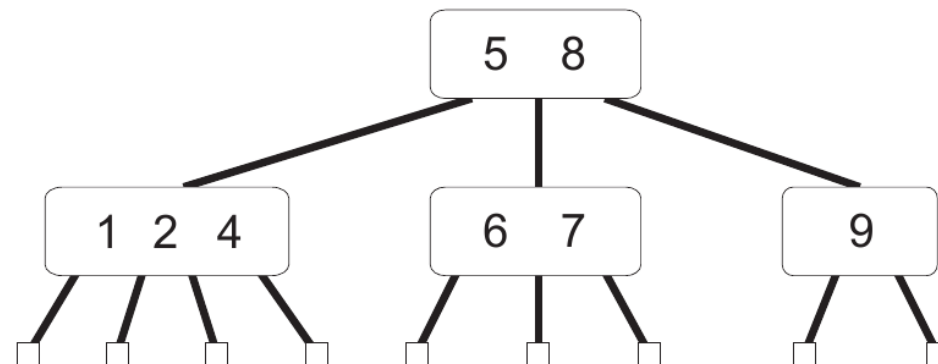


2-3-4 Bäume

- Ansatz
 - mehr als einen Schlüssel pro Knoten, hier 1 ... 3
 - entsprechend 2 ... 4 Kinder
 - Strukturänderungen sind begrenzt
- Ausgeglichenheit wird beim Einfügen sichergestellt.
- Implementierung durch Rot-Schwarz Bäume

2-3-4 Bäume

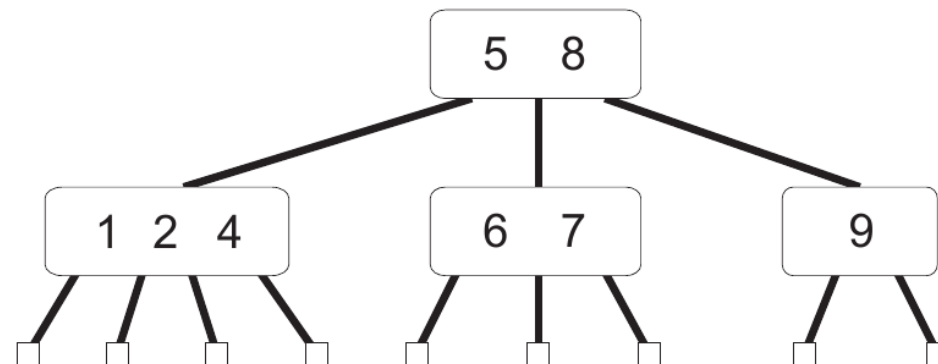
- Knotentypen:
 - 2-Knoten (binäre Knoten) mit einem Schlüssel
 - 3-Knoten mit bis zu 3 Nachfolgern und 2 Schlüsseln
 - 4-Knoten mit bis zu 4 Nachfolgern und 3 Schlüsseln
- Bei 3- und 4-Knoten: die Kante zwischen zwei Schlüsseln verweist auf einen Teilbaum mit Werten, die zwischen den beiden Schlüsseln liegen.





Suchen im 2-3-4 Baum

- Suche funktioniert wie Erweiterung der Suche im Binär-Baum
- Bei 3- und 4-Knoten Vergleich mit allen Schlüsselementen
- Dadurch Auswahl des Teilbaums unter den 2, 3 oder 4 Teilbäumen, in dem die Suche fortgesetzt wird.





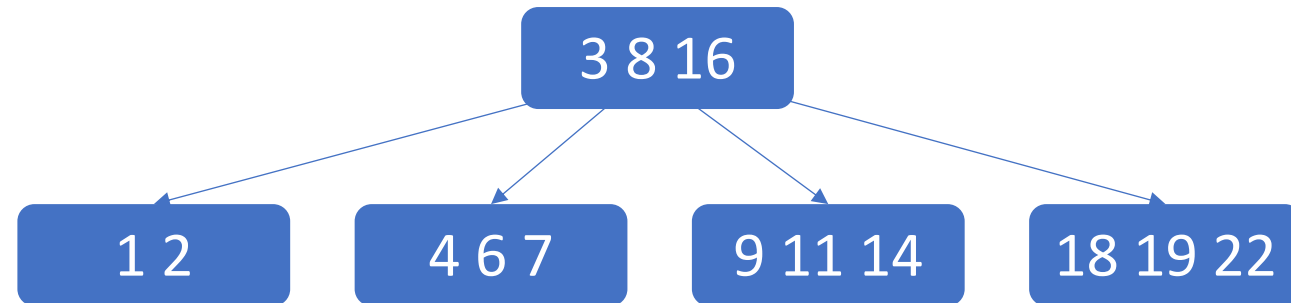
Einfügen im 2-3-4 Baum

- Zunächst wird durch eine (erfolglose) Suche die Einfügeposition ermittelt.
- Bei einem 2- oder 3-Knoten kann der Knoten einfach um das Element erweitert werden
- Bei einem 4-Knoten kann der Knoten nicht mehr erweitert werden, sondern muss aufgeteilt werden.
- Insgesamt haben wir 4 Schlüsselemente: Die 3 Schlüssel des 4-Knotens und das neue Schlüsselement.
- Zunächst wird der 4-Knoten geteilt: Aus dem 4-Knoten werden zwei 2-Knoten. Ein mittleres Schlüsselement wird zum Vorgängerknoten “nach oben” geschoben.
- Ist der Vorgängerknoten ebenfalls ein 4-Knoten, muss auch dieser Knoten geteilt werden, usw. (rekursiv oder iterativ)
- Währenddessen wird das neue Schlüsselement hinzugefügt.

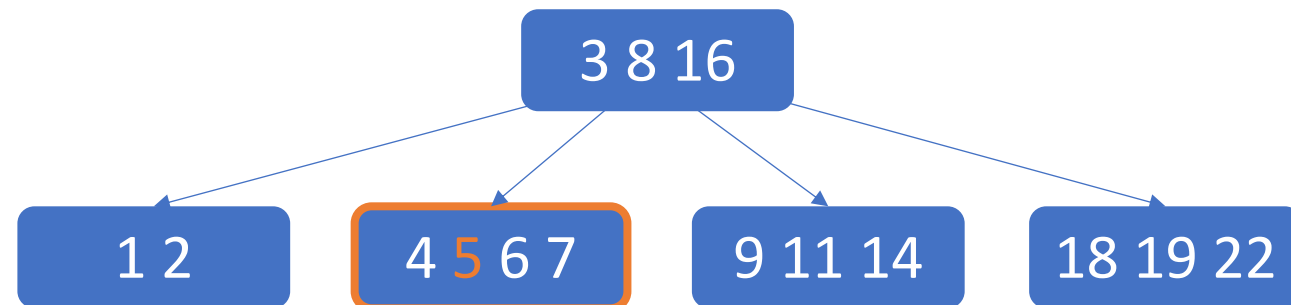


Beispiel: Einfügen im 2-3-4 Baum, Bottom-up

Original Baum



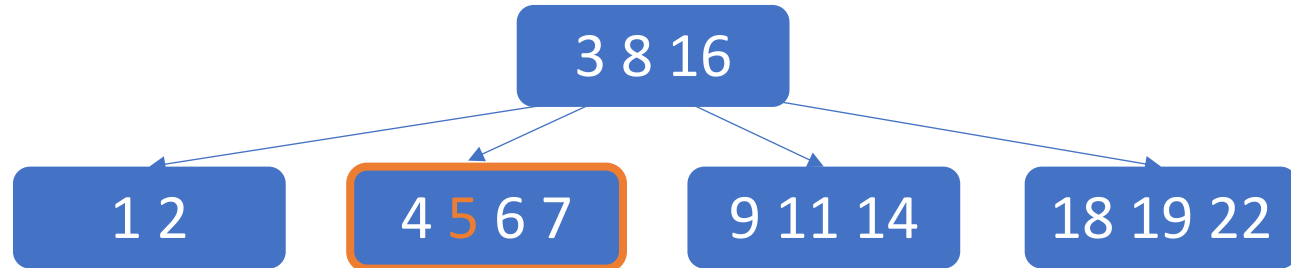
5 eingefügt:
nun vier Schlüssel



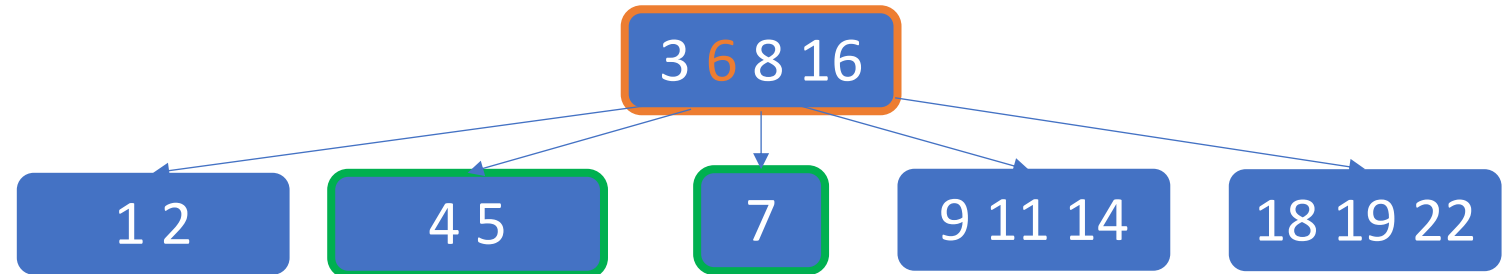


Beispiel: Einfügen im 2-3-4 Baum, Bottom-up

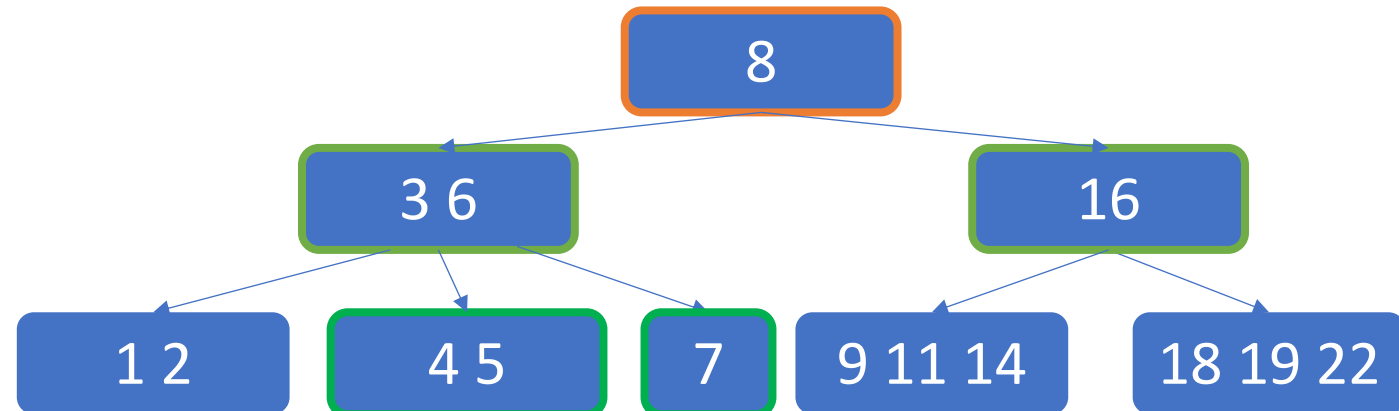
5 eingefügt:
nun vier Schlüssel



Knoten aufteilen,
ein Schlüssel nach oben



Knoten aufteilen,
ein Schlüssel nach oben





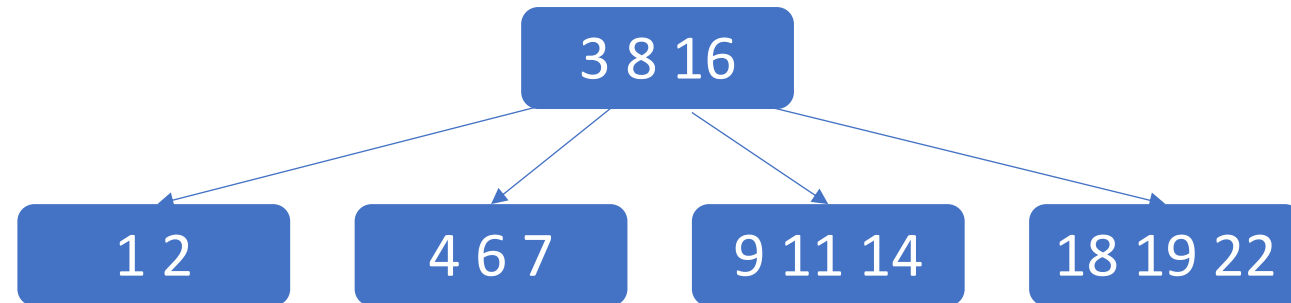
Einfügen im 2-3-4 Baum, Top-down

- Nachteil der Bottom-Up Methode:
 - Wir müssen im Baum nach oben navigieren
 - normalerweise keine „parent“-Zeiger vorhanden/nötig
- Top-Down Methode:
 - beim Suchen nach der Einfügeposition vorsorglich alle 4-Knoten aufteilen.
 - Dann geht das Aufteilen immer, weil die darüberliegenden Knoten maximal 3-Knoten sein können.
 - Das Einfügen geht dann auch immer, weil maximal ein 3-Knoten gefunden wird.

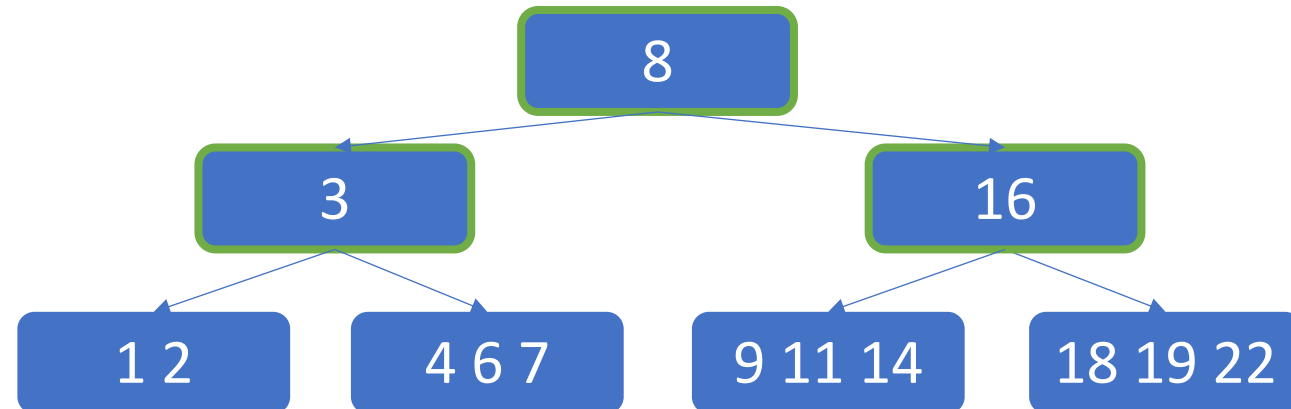


Beispiel: Einfügen im 2-3-4 Baum, Top-down

Original Baum



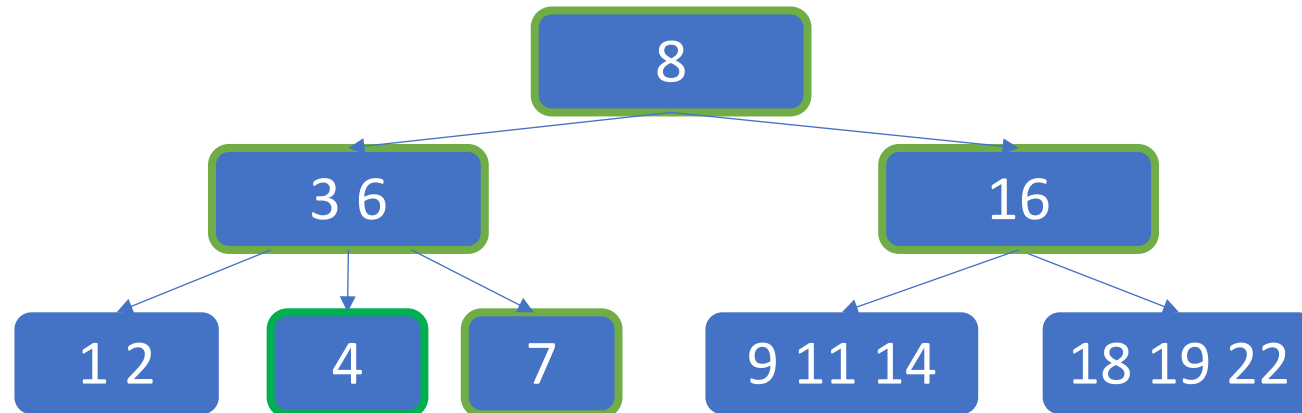
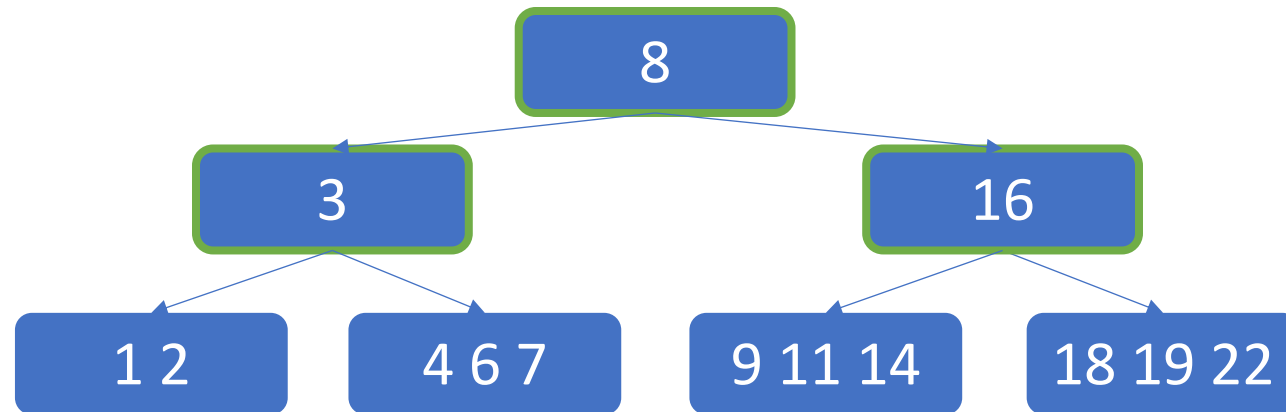
Suche Position für 5
Splitte 4-Knoten auf
dem Weg





Beispiel: Einfügen im 2-3-4 Baum, Top-down

Suche Position für 5
Splitte 4-Knoten auf
dem Weg

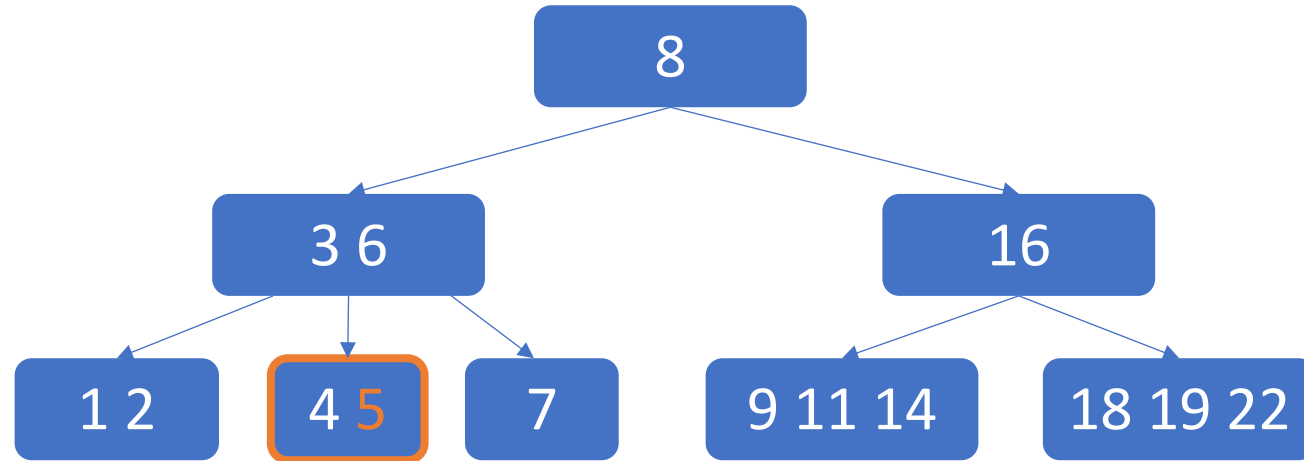




Beispiel: Einfügen im 2-3-4 Baum, Top-down

Suche Position für 5
Splitte 4-Knoten auf
dem Weg

5 einfügen: simpel





Übungsaufgabe

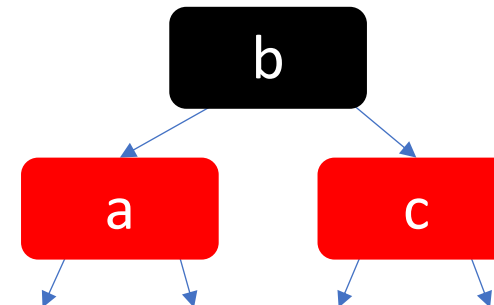
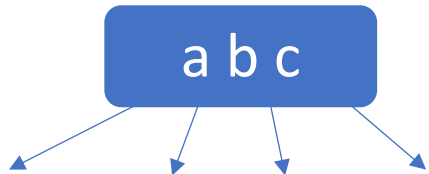
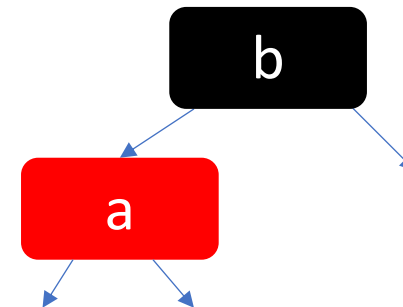
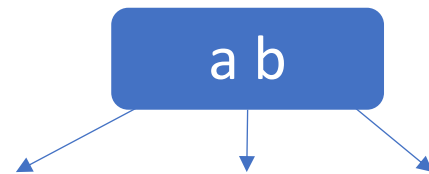
- Bauen Sie mit dem Top-Down Ansatz schrittweise einen 2-3-4 Baum auf, indem Sie nacheinander folgende Zahlen einfügen:
- 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 3, 5
- Ist der 2-3-4 Baum ausgeglichen?
- Sind die Blätter alle auf einem Niveau?



2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 3, 5

Rot-Schwarz Bäume

- Ausgleichen ist gute Eigenschaft des 2-3-4-Baums
- Verwaltung von Knoten mit unterschiedlicher Schlüsselanzahl ist umständlich
- Darstellung von 2-3-4-Bäumen als Binärbäume:

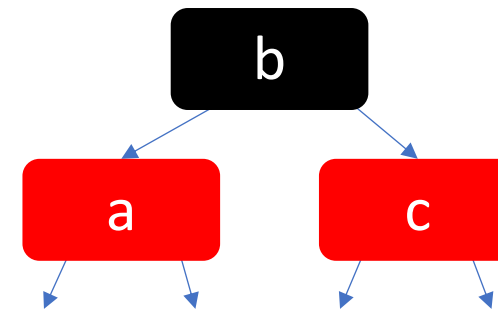
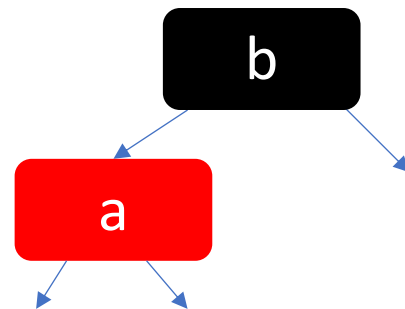






Definition Rot-Schwarz Bäume

- **Rot-Schwarz-Bäume** sind binäre Suchbäume, die die folgenden Kriterien, die sogenannten Rot-Schwarz-Eigenschaften, erfüllen:
 - Jeder Knoten ist entweder rot oder schwarz.
 - Jeder Null-Knoten ist per Definition schwarz.
 - Die Kinder jedes roten Knotens sind schwarz.
 - Für jeden Knoten k gilt: Jeder Pfad von k zu einem Blatt enthält die gleiche Anzahl schwarzer Knoten.





Operationen auf Rot-Schwarz Bäumen

- Da der Rot-Schwarz-Baum ein Binärbaum ist, können die **Suche** und das **Traversieren** für Binärbäume direkt verwendet werden.
- Beim Einfügen müssen die Rot-Schwarz-Eigenschaften eingehalten werden, falls Knoten gesplittet werden.
- Wir realisieren das **Einfügen** bei Rot-Schwarz-Bäumen mit **Splitten** auf dem Weg von der Wurzel zur Einfügeposition (**Top-Down**)



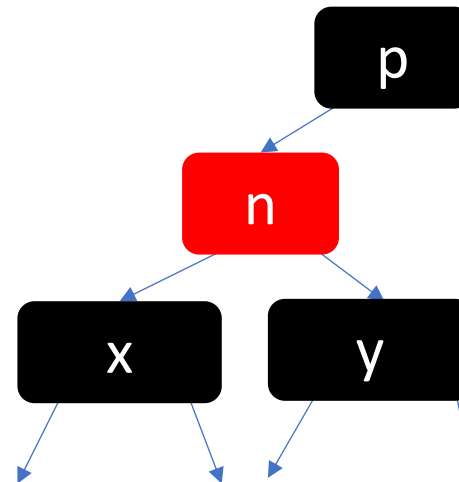
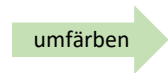
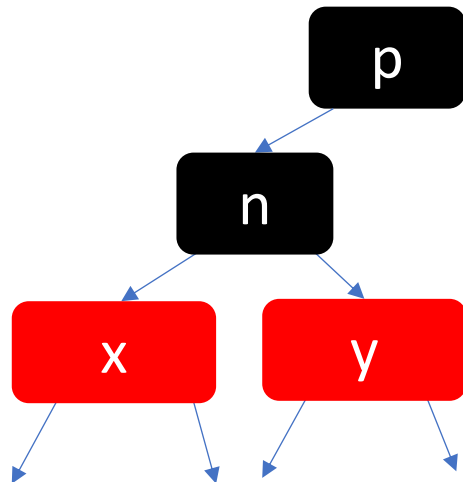
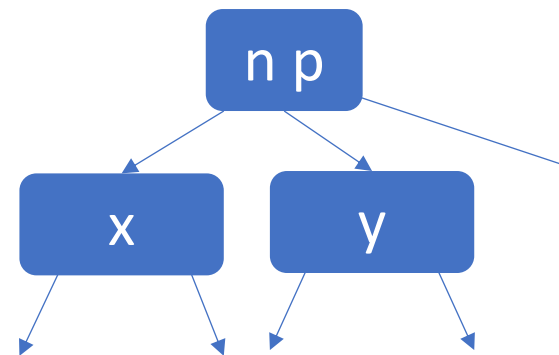
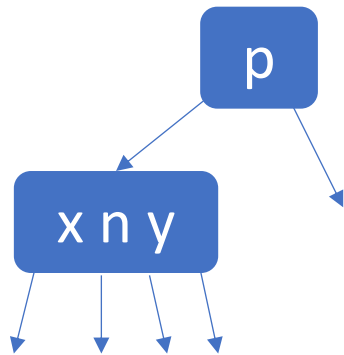
Einfügen in Rot-Schwarz Bäumen

- Splitten der 4-Knoten: top-down, nichtrekursiv
- zwei Hauptfälle unterscheiden
 - 1. der 4-Knoten hängt an einem 2-Knoten
 - 2. der 4-Knoten hängt an einem 3-Knoten
- Unterfälle (2.):
 - A Teilbaum hängt links
 - B Teilbaum hängt in der Mitte
 - C Teilbaum hängt rechts
- Bezeichnungen:
 - n aktuell betrachteter Knoten (node)
 - p dessen Elternknoten (parent)
 - g dessen Elternknoten von p (grandparent)



Splitten in Rot-Schwarz Bäumen: Fall 1

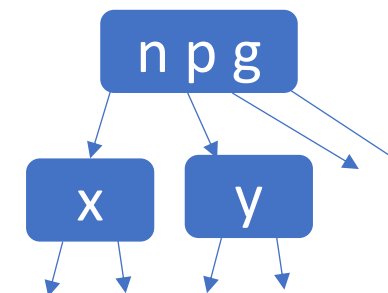
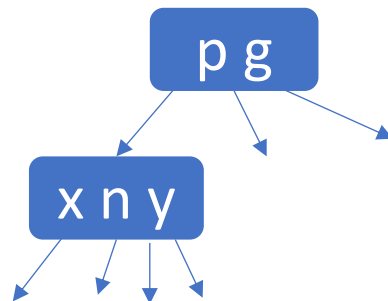
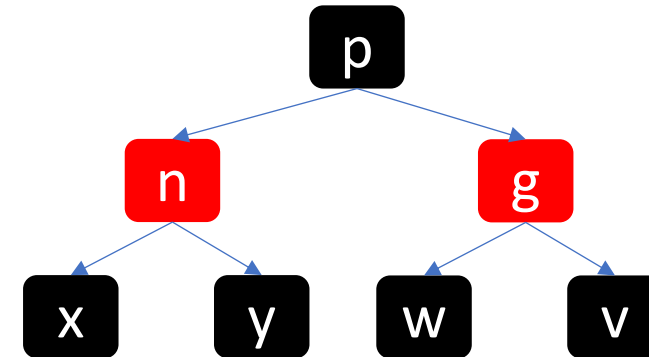
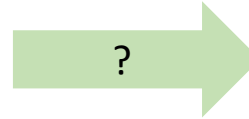
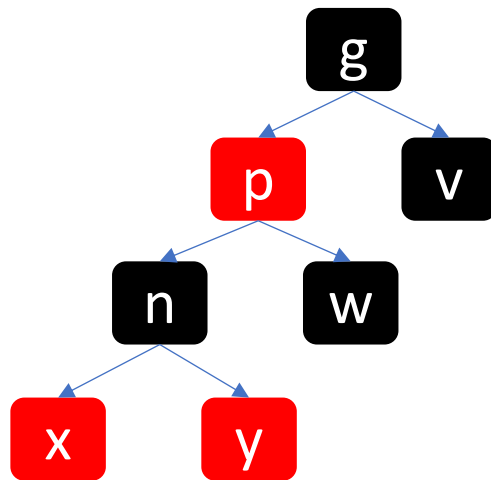
- Fall 1: 4 Knoten soll gesplittet werden und hängt an 2 Knoten





Splitten in Rot-Schwarz Bäumen: Fall 2A

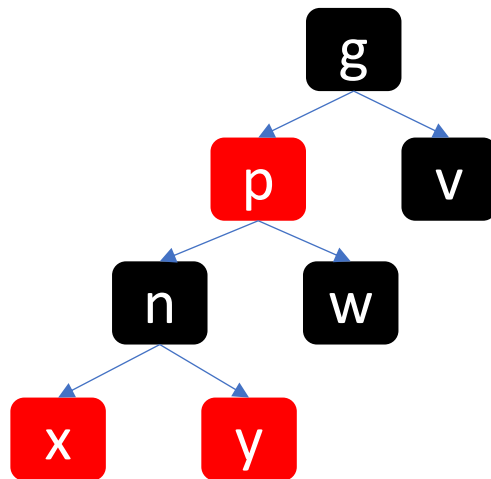
- Fall 2A: 4 Knoten soll gesplittet werden und hängt an 3 Knoten links außen



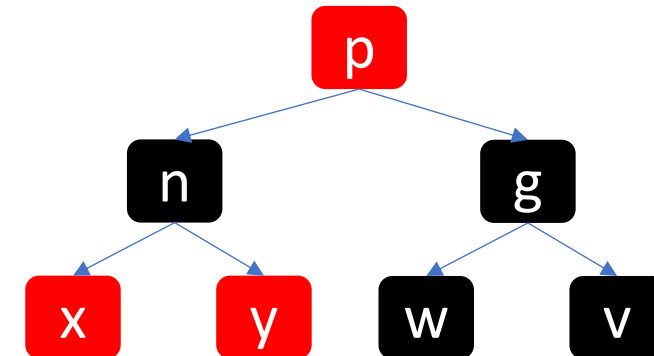


Splitten in Rot-Schwarz Bäumen: Fall 2A

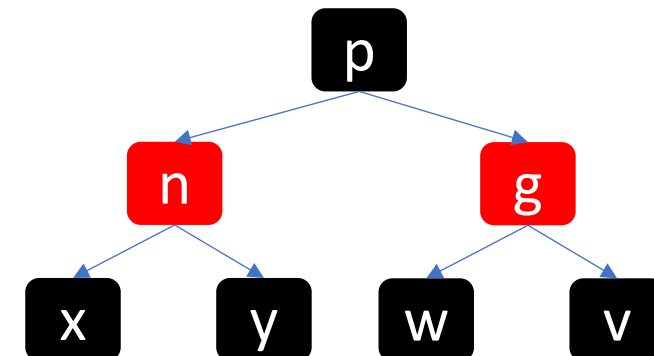
- Fall 2A: 4 Knoten soll gesplittet werden und hängt an 3 Knoten links außen



Rotation



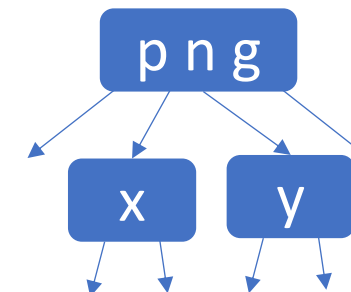
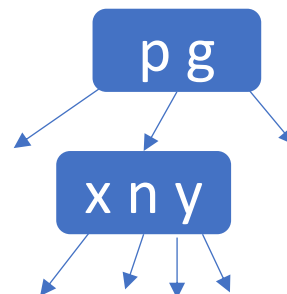
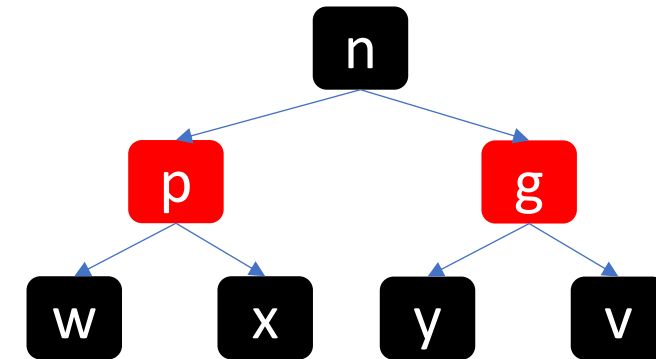
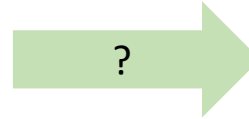
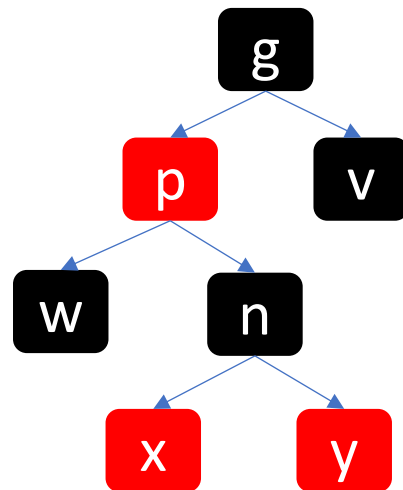
umfärben





Splitten in Rot-Schwarz Bäumen: Fall 2B

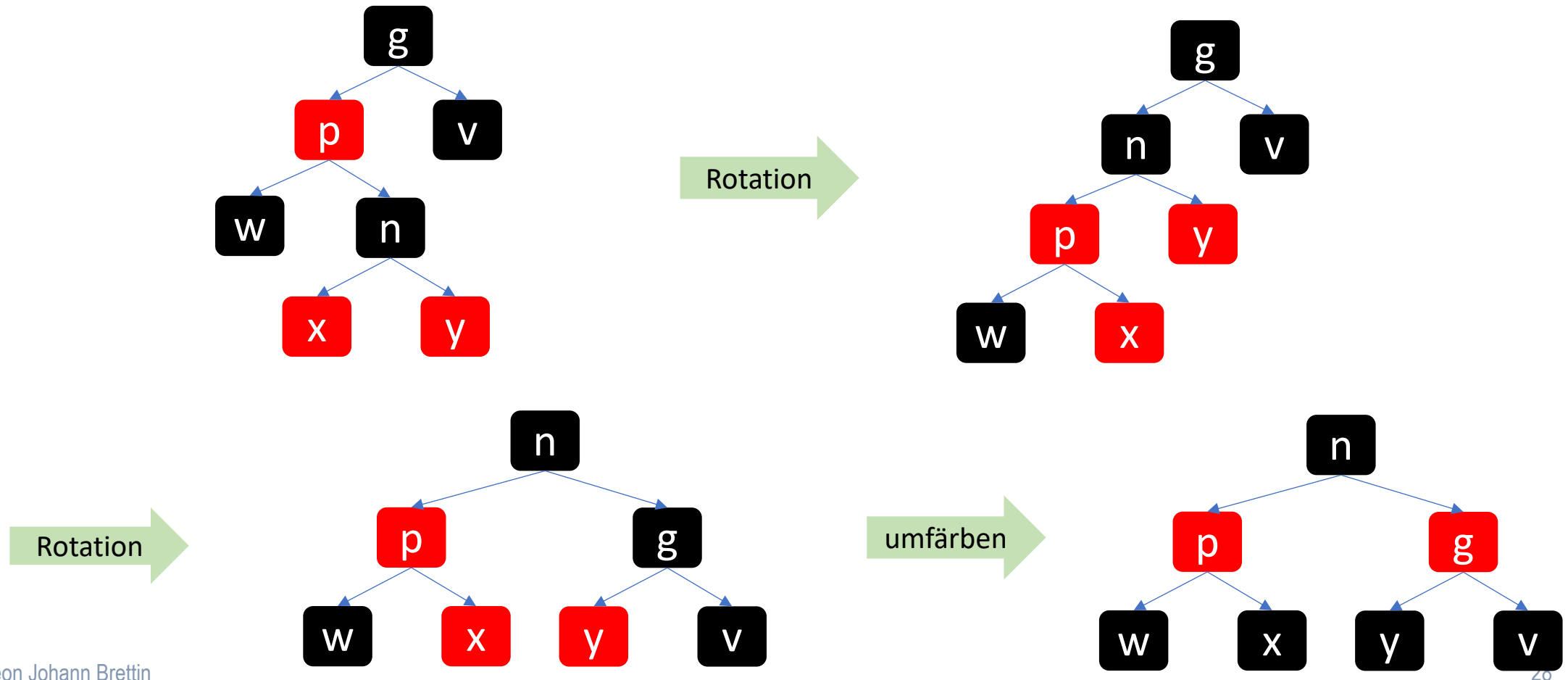
- Fall 2B: 4 Knoten soll gesplittet werden und hängt an 3 Knoten in der Mitte





Splitten in Rot-Schwarz Bäumen: Fall 2B

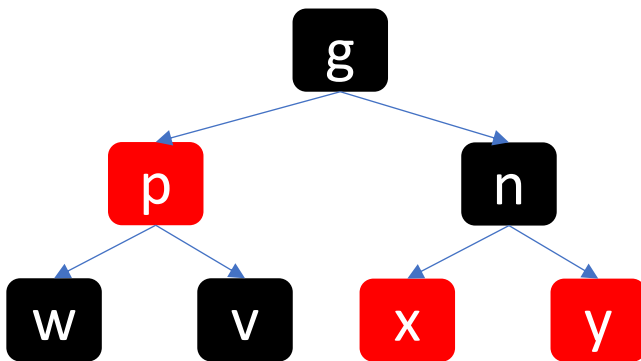
- Fall 2B: 4 Knoten soll gesplittet werden und hängt an 3 Knoten in der Mitte



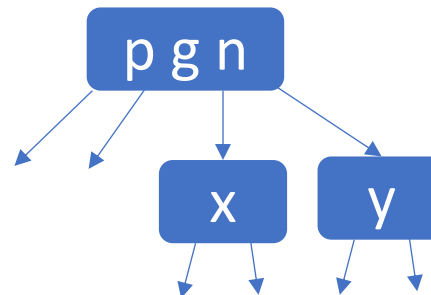
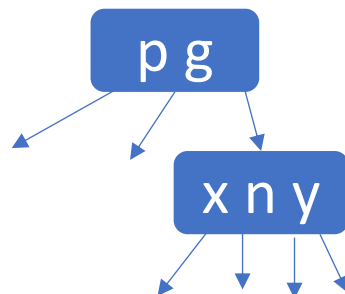
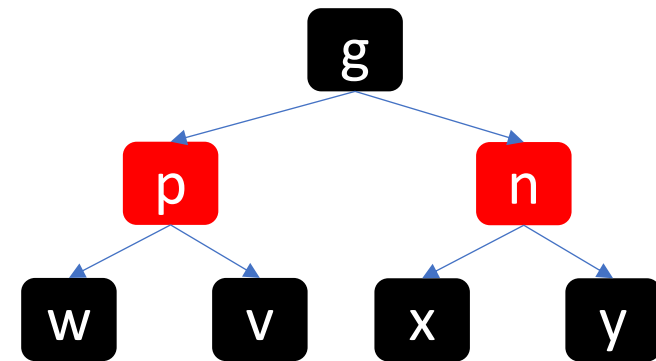


Splitten in Rot-Schwarz Bäumen: Fall 2C

- Fall 2C: 4 Knoten soll gesplittet werden und hängt an 3 Knoten rechts außen



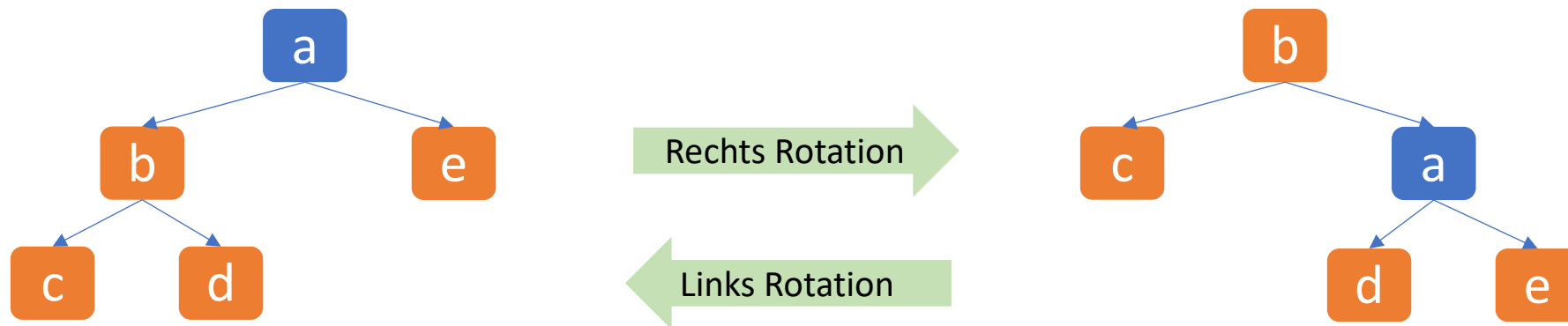
umfärben





Rotation in Suchbäumen

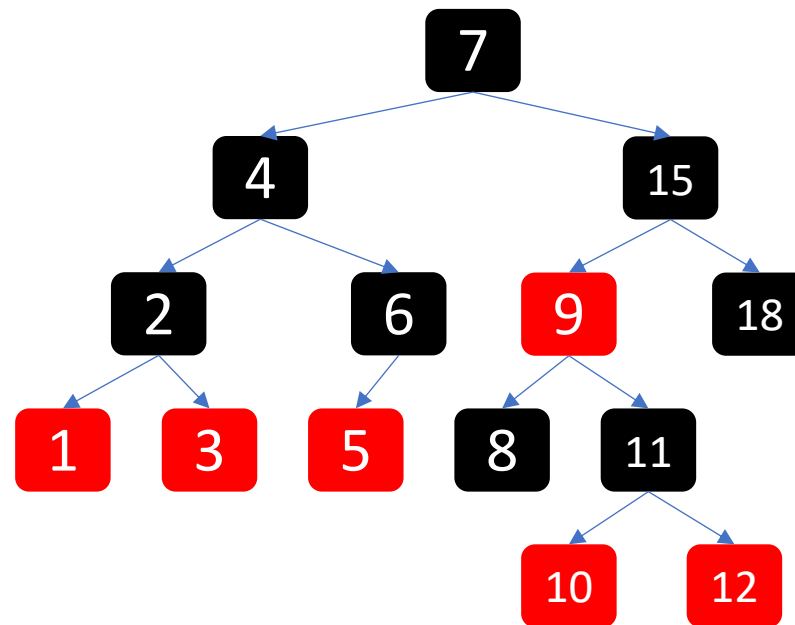
- Realisierung des Splittens von Knoten durch die Operationen Umfärben und Rotation.
- Rotationen verletzen die Suchbaumeigenschaft nicht.
- Komplexität: $O(1)$





Beispiel: Einfügen von 13

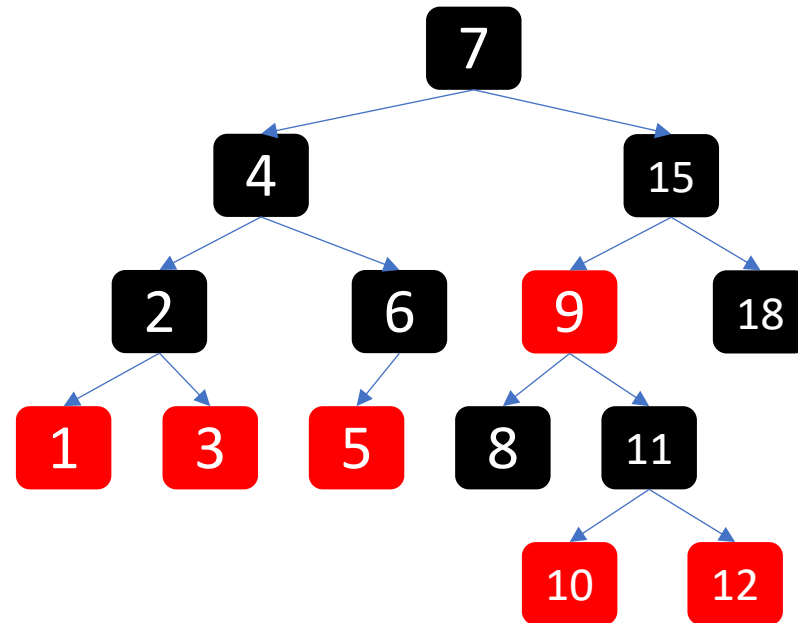
- Top-down: Einfügeposition suchen, dabei alle 4 Knoten splitten.





Beispiel: Einfügen von 13

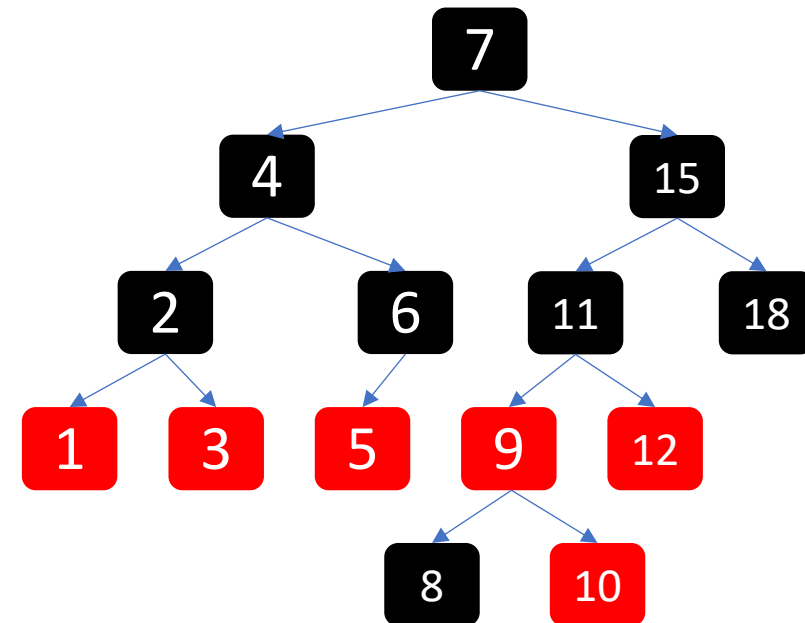
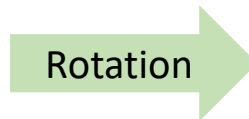
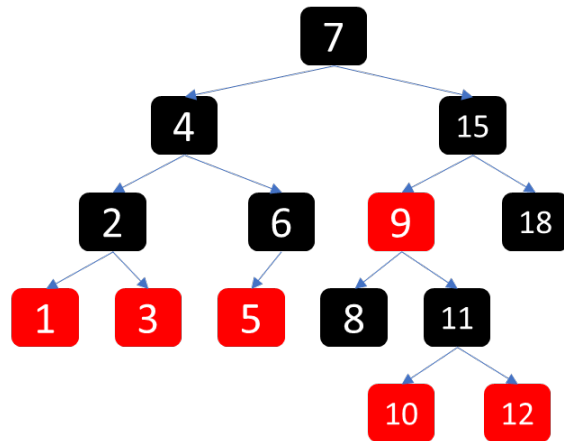
- 11 ist ein 4 Knoten, Fall 2B





Beispiel: Einfügen von 13

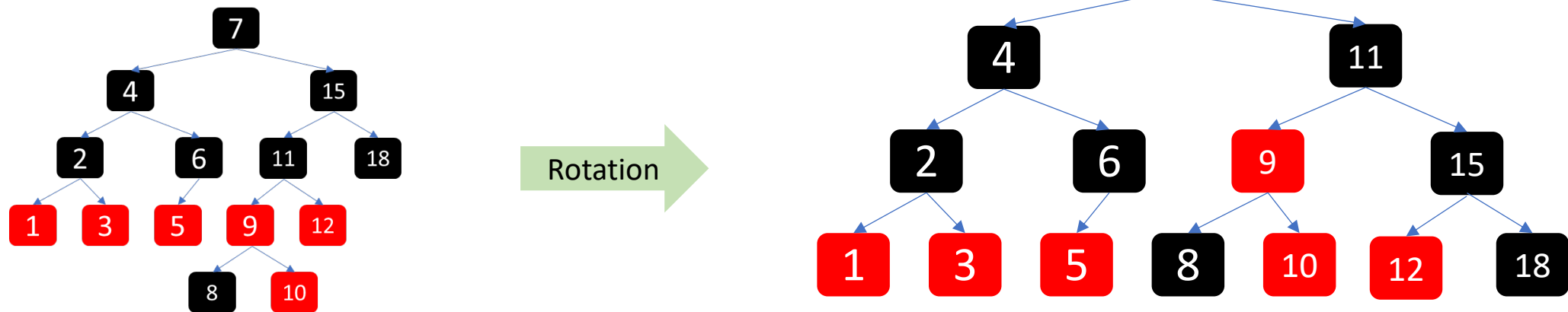
- 2-mal rotieren
- Umfärben





Beispiel: Einfügen von 13

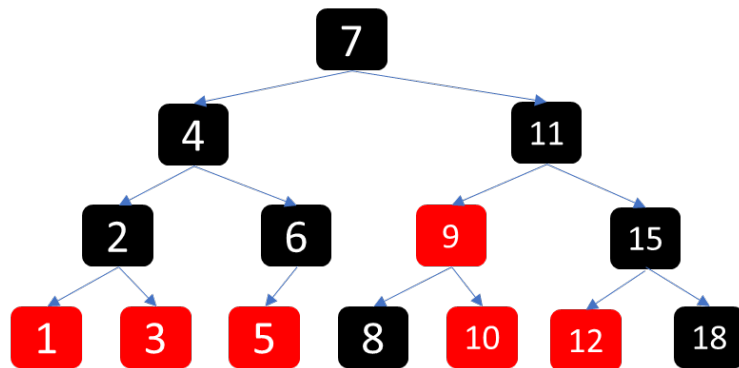
- 2-mal rotieren
- Umfärben



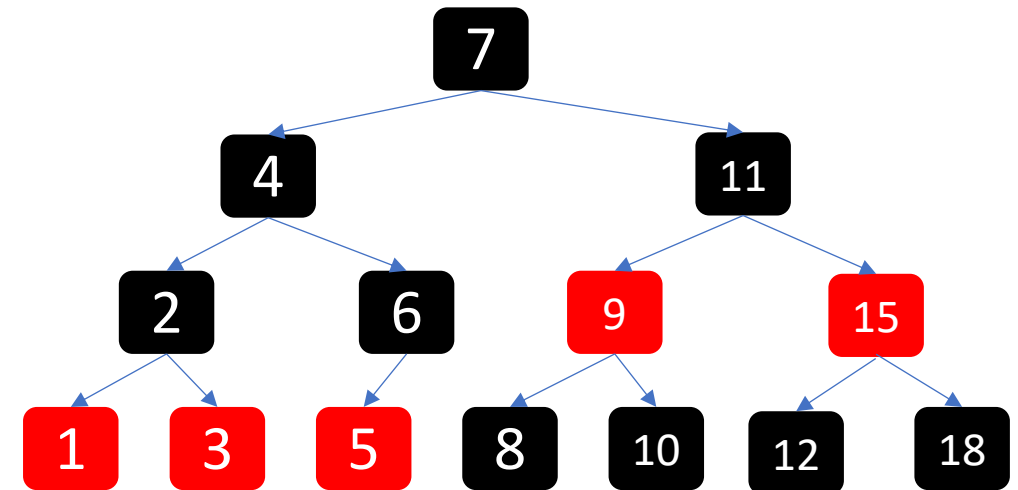


Beispiel: Einfügen von 13

- 2-mal rotieren
- **Umfärben**



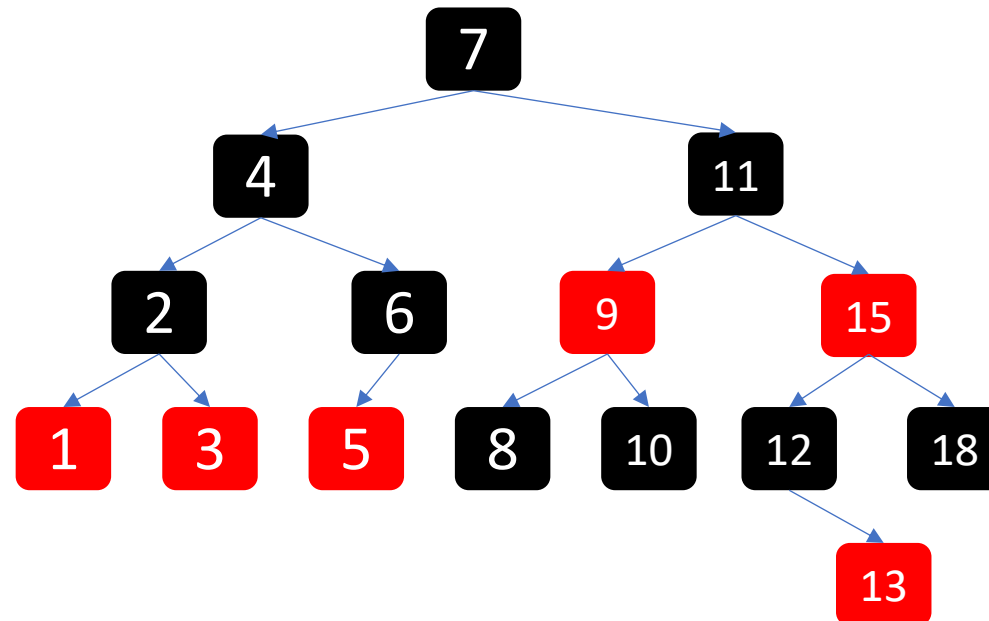
umfärben





Beispiel: Einfügen von 13

- Weiter nach Position für 13 suchen!
- 13 Einfügen





Komplexität

- Höhe des ausgeglichenen Baums: $h = \log_2 n$
- Höhe des Rot-Schwarz-Baums: $h \leq 2 \log_2 (n + 1)$
(ohne Beweis)
- Damit Komplexität der Operationen Suche, Einfügen im Rot-Schwarz Baum $O(\log n)$.
- Die zusätzlichen Rotationen, die wir durchführen, haben die Komplexität $O(1)$.

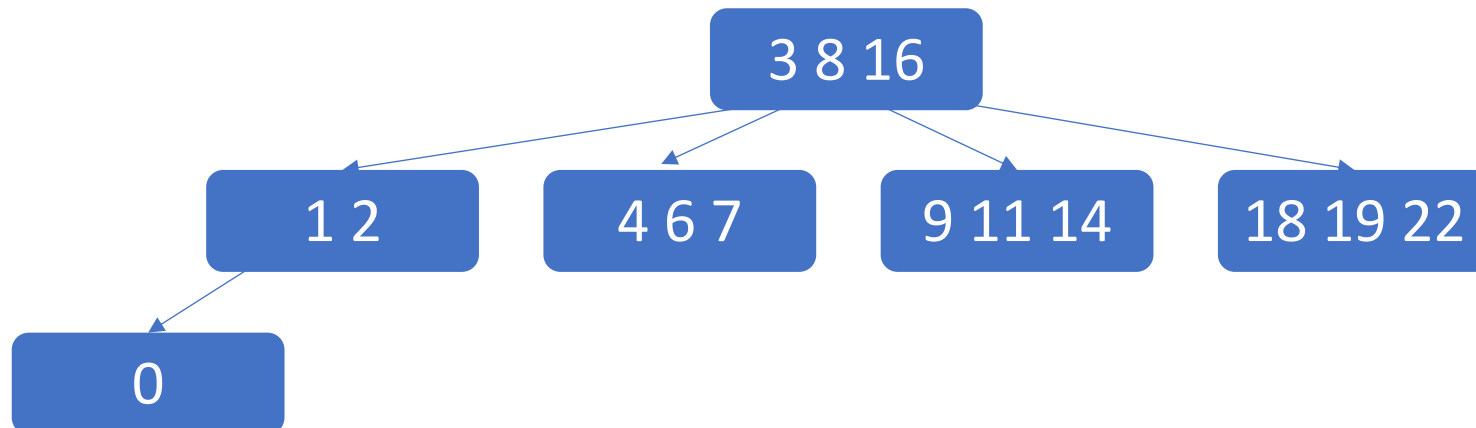
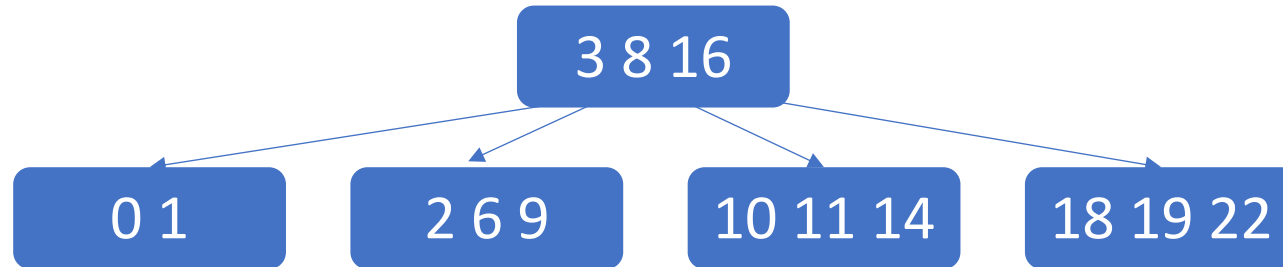


Zusammenfassung Bäume

- Binäre Bäume
- Ausgeglichene Bäume
 - Rot-Schwarz Bäume
 - Suchen, Einfügen, Löschen $O(\log n)$
 - AVL-Bäume
 - Suchen etwas schneller; Einfügen, Löschen etwas langsamer, aber $O(\log n)$
 - B-Bäume
 - Für externe Speicher (Datenbanken)
 - interne Operationen (Umkopieren) schnell.

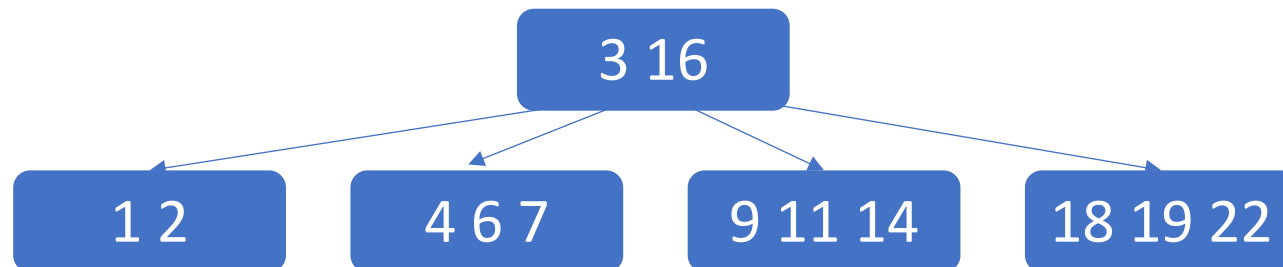
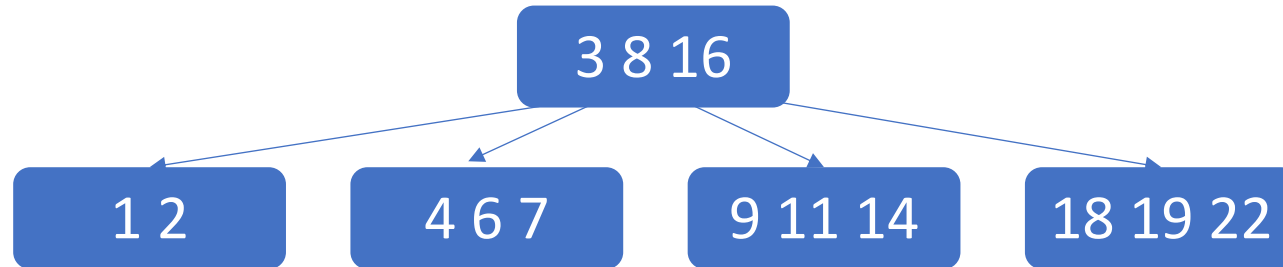


Welches ist ein legaler 2-3-4-Baum?



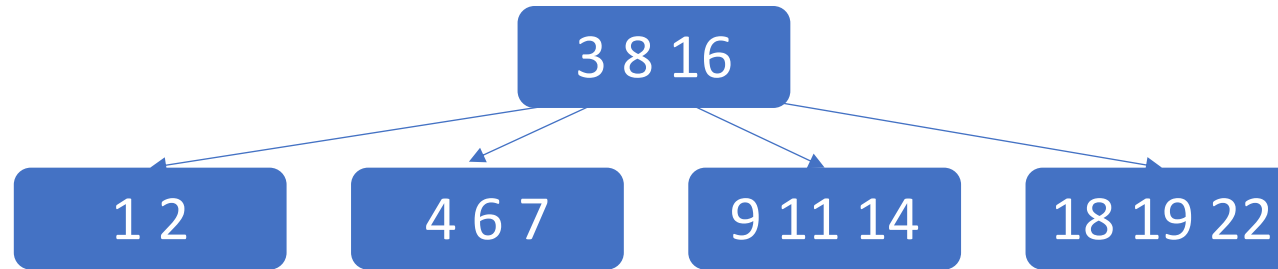


Welches ist ein legaler 2-3-4-Baum?





Welche Schlüssel werden bei der Suche nach der 18 betrachtet?



- A. 3,8,16,19,18
- B. 1,2,3,4,6,7,8,9,11,14,16,18,19,22
- C. 8,16,19,18
- D. 8,19,18
- E. 3,8,16,18

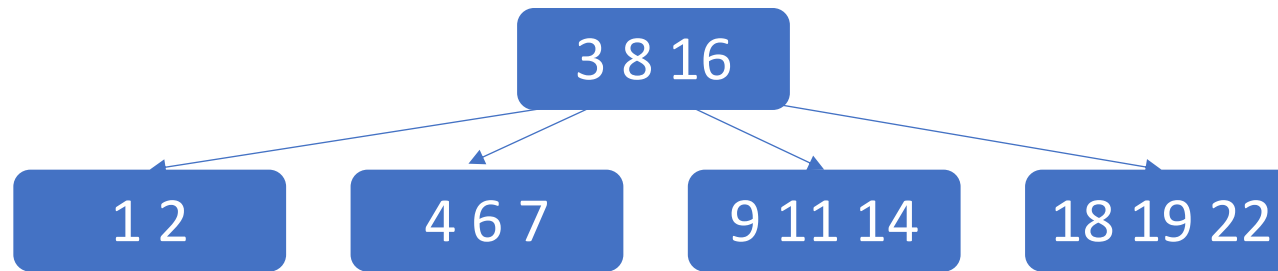


Welche Aussage ist falsch?

- Beim Einfügen im 2-3-4-Baum
 - A. Landet das neue Element immer in einem Blattknoten
 - B. Erfolgt bei 2- und 3-er Knoten keine Strukturänderung
 - C. Muss ein 4-er-Knoten aufgespalten werden
 - D. Wird nur die Blatt-Ebene verändert
 - E. Reicht Aufwand $O(\log n)$.



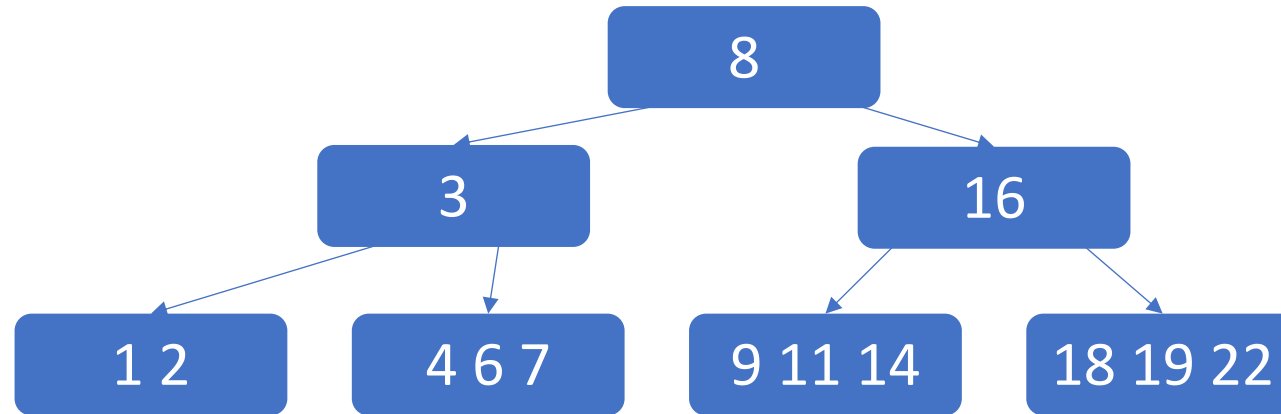
Einfügen von 25 im 2-3-4-Baum, top-down



- Welcher Knoten wird zuerst aufgespalten?
 - A. 1 2
 - B. 4 6 7
 - C. 9 11 14
 - D. 18 19 22
 - E. 3 8 16



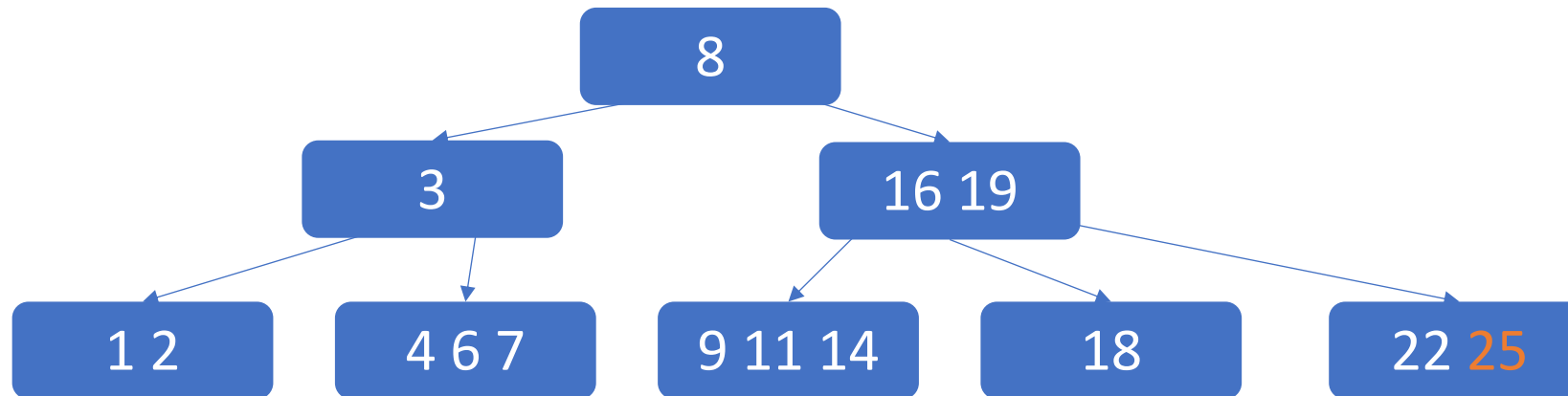
Einfügen von 25 im 2-3-4-Baum, top-down



- Welcher Knoten wird zuerst aufgespalten?
 - A. 16
 - B. 9 11 14
 - C. 18 19 22
 - D. 22 25

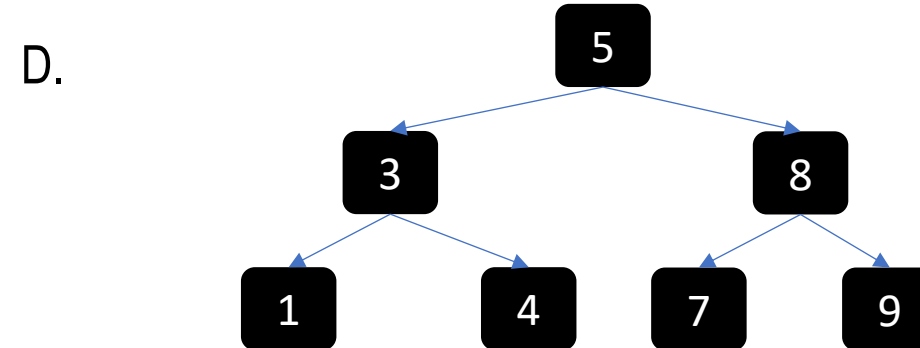
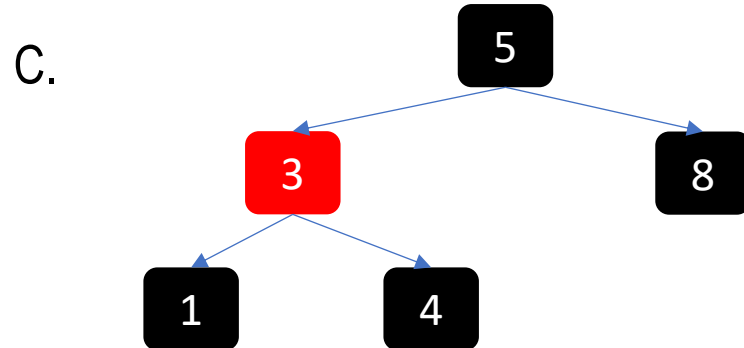
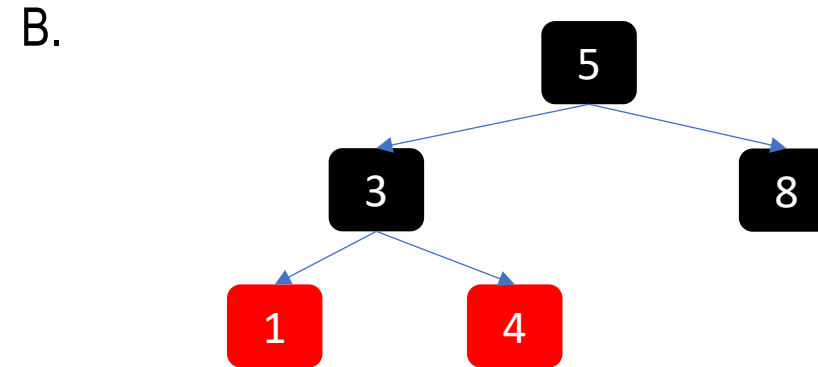
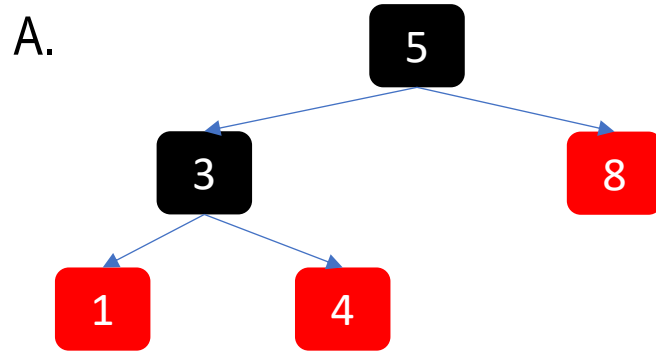


Einfügen von 25 im 2-3-4-Baum, top-down





Welches ist kein legaler Rot-Schwarz Baum?





Einfügen in Rot-Schwarz Bäumen

- Splitten der 4-Knoten: top-down, nichtrekursiv
- Hauptfälle unterscheiden
 - 1. der 4-Knoten hängt an einem 2-Knoten
 - 2. der 4-Knoten hängt an einem 3-Knoten
 - 3. der 4-Knoten hängt an einem 4-Knoten
- Unterfälle:
 - A Teilbaum hängt links
 - B Teilbaum hängt in der Mitte
 - C Teilbaum hängt rechts
- Welcher Fall ist unmöglich?
 - A: 1 , B: 2, C: 3, D: 2A, E: 2C



Einfügen in Rot-Schwarz Bäumen

- Splitten der 4-Knoten: top-down, nichtrekursiv
- Hauptfälle unterscheiden
 - 1. der 4-Knoten hängt an einem 2-Knoten
 - 2. der 4-Knoten hängt an einem 3-Knoten
 - 3. der 4-Knoten hängt an einem 4-Knoten
- Unterfälle:
 - A Teilbaum hängt links
 - B Teilbaum hängt in der Mitte
 - C Teilbaum hängt rechts
- Welcher Fall erfordert nur Umfärben?
 - A: 1 , B: 2A, C: 2B, D: 2C, E: mehr als 1 Fall.



Einfügen in Rot-Schwarz Bäumen

- Splitten der 4-Knoten: top-down, nichtrekursiv
- Hauptfälle unterscheiden
 - 1. der 4-Knoten hängt an einem 2-Knoten
 - 2. der 4-Knoten hängt an einem 3-Knoten
 - 3. der 4-Knoten hängt an einem 4-Knoten
- Unterfälle:
 - A Teilbaum hängt links
 - B Teilbaum hängt in der Mitte
 - C Teilbaum hängt rechts
- Welcher Fall erfordert eine Rotation?
 - A: 1 , B: 2A, C: 2B, D: 2C, E: mehr als 1 Fall.



Einfügen in Rot-Schwarz Bäumen

- Splitten der 4-Knoten: top-down, nichtrekursiv
- Hauptfälle unterscheiden
 - 1. der 4-Knoten hängt an einem 2-Knoten
 - 2. der 4-Knoten hängt an einem 3-Knoten
 - 3. der 4-Knoten hängt an einem 4-Knoten
- Unterfälle:
 - A Teilbaum hängt links
 - B Teilbaum hängt in der Mitte
 - C Teilbaum hängt rechts
- Welcher Fall erfordert zwei Rotationen?
 - A: 1 , B: 2A, C: 2B, D: 2C, E: mehr als 1 Fall.



Welche Aussage ist korrekt?

- A. Die Höhe des Rot-Schwarz-Baumes ist $O(n)$
- B. Die Höhe des Rot-Schwarz-Baumes ist $O(\log n)$
- C. In einem Rot-Schwarz-Baum gibt es mindestens so viele rote Knoten, wie schwarze Knoten.
- D. Das Traversieren eines Rot-Schwarz-Baumes erfordert $O(n \log n)$ Operationen
- E. Die Operation Einfügen erfordert $O(n)$, das Suchen $O(\log n)$ Operationen.